



LangGraph

Contents

Langchain

01. Lang Graph 란?

02. 조건부 분기와 루프 구현

03. tool 활용하기

04. Human in the loop

01.

Lang Graph 란?

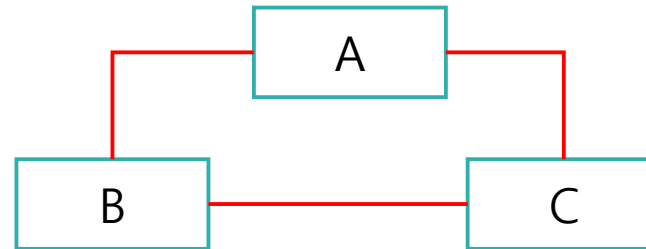
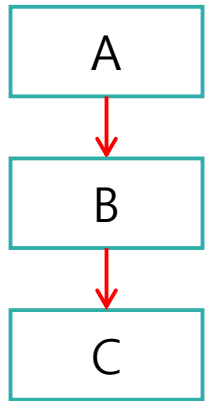
Lang Graph 란?

✓ Lang graph 의 정의

- langChain 개발사에서 만든 LLM 기반의 복잡한 에이전트 워크플로우를 구축, 조율, 관리를 위한 오픈소스 프레임워크

✓ Langchain 과의 차이점

- langChain 은 한 방향 구조로 되돌릴 수 없다.
- langGraph 는 순환 구조로 에러 또는 기준에 미치지 못하는 결과가 있으면 되돌아가서 다시 수행 할 수 있다.



Lang Graph 란?

Lang graph 의 핵심 3 요소

요소	설명
State(상태)	그래프를 관통하며 공유되는 데이터 저장소 TypedDict 또는 Pydantic 모델로 정의하며, 각 노드가 반환하는 값에 의해 누적 및 업데이트
Nodes(노드)	특정 작업(LLM 호출, 파이썬 함수 실행 등)을 수행하는 실행 단위 현재의 State를 입력받아 새로운 State의 일부를 반환
Edges(엣지)	노드와 노드를 연결하는 경로 일반 엣지 - 무조건 다음 노드로 이동하는 조건부 엣지 - 조건부 함수에 따라 동적으로 방향을 결정

01_basic.py

01_basic_stream.py

Lang Graph 란?

✓ Lang graph 의 핵심 3 요소

- workflow 의 등록 과정을 보면 전반적인 흐름을 이해 할 수 있다.

```
work_flow = StateGraph(SimpleState)
```

공통으로 사용할 상태 객체 지정

```
work_flow.add_node("refiner", refine_text_node)  
work_flow.add_node("generator", call_llm_node)
```

"refiner" 라는 별칭으로 refine_text_node 함수 등록
"generator" 라는 별칭으로 call_llm_node 함수 등록

```
work_flow.set_entry_point("refiner")
```

workflow 의 시작은 별칭 refiner

```
work_flow.add_edge("refiner", "generator")
```

"refiner" 다음은 "generator" 로 이동

```
work_flow.add_edge("generator", END)
```

"generator" 다음은 종료(END) 한다.

02.

조건부 분기와 루프 구현

조건부 분기와 루프 구현

☑ 조건부 분기 구현

- 조건부 라우팅은 AI 답변의 결과에 따라 다음 행동을 스스로 결정할 수 있다.
- Condition Edge 는 AI 판단에 따라 분기를 처리 할 수 있다.

```
# 시작점(Entry Point) 설정
workflow.set_entry_point("analyzer")

# 결과에 따른 조건부 라우팅 설정
workflow.add_conditional_edges(
    "analyzer", # 1. 출발 노드 지정
    route_by_department, # 2. 다음 노드를 결정할 라우터 함수(분기)
    {
        "go_to_billing": "billing_agent", # 3. 라우터 반환값이 "go_to_billing"이면 billing_agent로 이동
        "go_to_tech": "tech_agent" # 3. 라우터 반환값이 "go_to_tech"이면 tech_agent로 이동
    }
)

# 일반 엣지 설정 (각 부서 처리가 끝나면 워크플로우 종료)
workflow.add_edge("billing_agent", END)
workflow.add_edge("tech_agent", END)
```

조건부 분기와 루프 구현

조건부 분기 구현

- 아래는 실행할 노드 들이다.

```
def billing_node(state: SupportState) -> Dict:
    print("[Billing Dept] 결제 관련 문의를 처리 중...")
    return {"response": "결제 및 환불 관련 안내: 마이페이지 > 결제 내역에서 신청하실 수 있습니다."}

def tech_node(state: SupportState) -> Dict:
    print("[Tech Dept] 기술 지원 문의를 처리 중...")
    return {"response": "기술 지원 안내: 서비스 이용에 불편을 드려 죄송합니다."}
```

02_condition_edge.py

조건부 분기와 루프 구현

☑ 조건부 분기 구현

- 각 노드들은 수행 기능이 있고 이후에 Graph 에 등록이 된다.

```
def analyzer_node(state: SupportState) -> Dict:  
    print("[Analyzer] 문의 내용을 분석 중...")  
    prompt = f"""당신은 고객 문의 분류기입니다.  
...(중략)...  
고객 문의: {state['query']}  
분류 결과:"""  
  
    res = llm.invoke(prompt)  
    dept = res.content.strip().lower()  
  
    return {"department": dept}
```

analyzer_node 는 고객 문의를 통해 어느 노드로 이동할지 (billing, technical) 반환한다.

02_condition_edge.py

조건부 분기와 루프 구현

☑ 조건부 분기 구현

- 라우터 함수는 state(공통 저장소) 값에 따라 이동할 노드 이름을 반환한다.

```
def route_by_department(state: SupportState) -> str:  
    if state["department"] == "billing":  
        return "go_to_billing"  
    else:  
        return "go_to_tech"
```

State 값이 billing 인지 여부에 따라 go_to_billing 또는 go_to_tech 를 반환 한다.

조건부 분기와 루프 구현

☑ 조건부 분기 구현

- 이제 조건부 분기가 어떻게 이루어지는지 순서를 살펴보자

(1) 이 workflow 에서 사용할 공통 저장소를 등록 한다.

```
workflow = StateGraph(SupportState)
```

```
class SupportState(TypedDict):  
    query: str  
    department: str  
    response: str
```

(2) 각 지정된 이름으로 노드들을 등록 한다.

```
workflow.add_node("analyzer", analyzer_node)  
workflow.add_node("billing_agent", billing_node)  
workflow.add_node("tech_agent", tech_node)
```

```
def billing_node(state: SupportState) -> Dict:  
  
def tech_node(state: SupportState) -> Dict:  
  
def analyzer_node(state: SupportState) -> Dict:
```

조건부 분기와 루프 구현

☑ 조건부 분기 구현

- 이제 조건부 분기가 어떻게 이루어지는지 순서를 살펴보자

(3) 등록된 노드 중에 analyzer 함수를 가장먼저 실행 하게 된다.

```
workflow.set_entry_point("analyzer")
```

```
def analyzer_node(state: SupportState) -> Dict:
```

analyzer_node 는 이동할 노드명(billing, technical) 을 반환한다.

(4) Analyzer 를 실행해서 나온 값을 route_by_department 에 전달 된다.(State 에 담겨져서)

(5) 전달된 값에 따라 go_to_billing 또는 go_to_tech 를 반환한다.

(6) 이 값에 따라 이동할 노드 명이 지정되고 등록된 노드로 향한다.

```
workflow.add_conditional_edges(
```

```
    "analyzer",  
    route_by_department,
```

```
    {  
        "go_to_billing": "billing_agent",  
        "go_to_tech": "tech_agent"  
    }  
)
```

```
def route_by_department(state: SupportState) -> str:
```

```
    if state["department"] == "billing":
```

```
        return "go_to_billing"
```

```
    else:
```

```
        return "go_to_tech"
```

```
workflow.add_node("billing_agent", billing_node)
```

```
workflow.add_node("tech_agent", tech_node)
```

조건부 분기와 루프 구현

조건부 분기 구현

- 이제 조건부 분기가 어떻게 이루어지는지 순서를 살펴보자

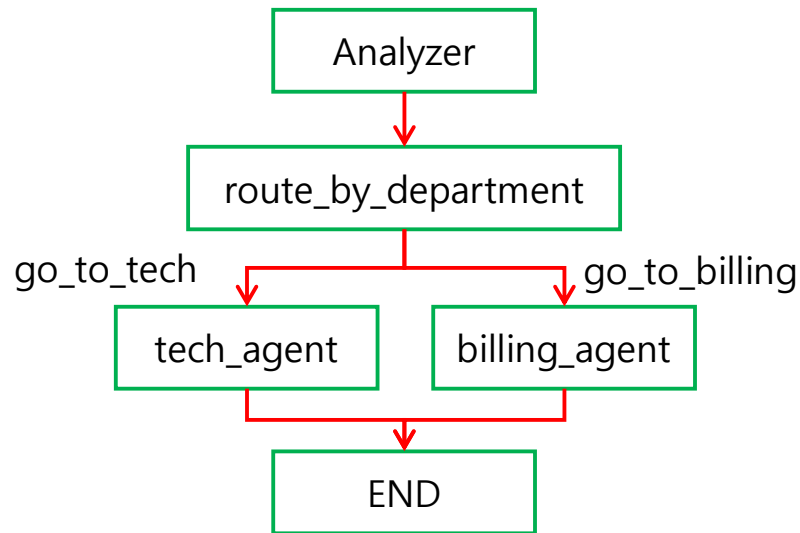
(7) 해당 노드를 진행하고 나면 다른 이동 없이 종료(END) 하게 된다.

```
workflow.add_edge("billing_agent", END)  
workflow.add_edge("tech_agent", END)
```

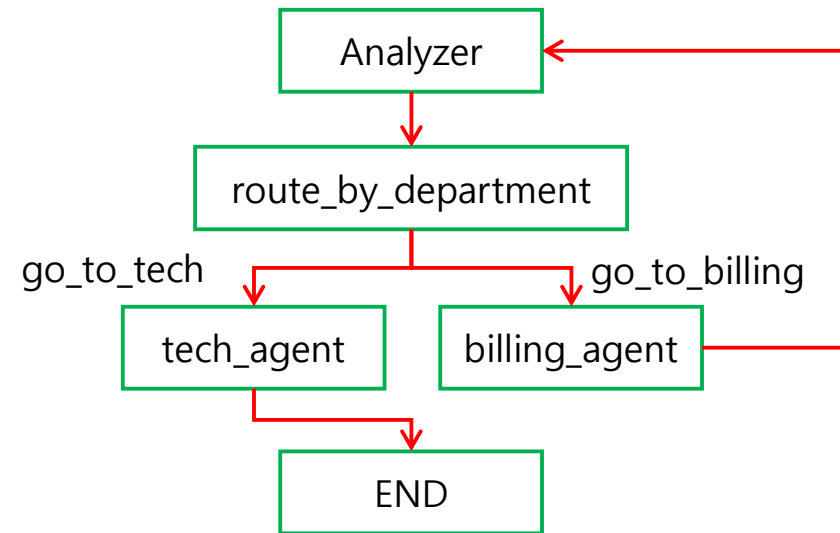
조건부 분기와 루프 구현

루프 구현

- 이제 조건부 분기를 도식해 보면 다음과 같다.
- 하지만 만약 billing_agent 가 END 가 아니라면 어떻게 될지 그려보자
- 이렇게 특정 조건이 발생했을때 특정 노드로 돌려보냄으로써 loop 를 구현 할 수 있다.



```
workflow.add_edge("billing_agent", END)  
workflow.add_edge("tech_agent", END)
```

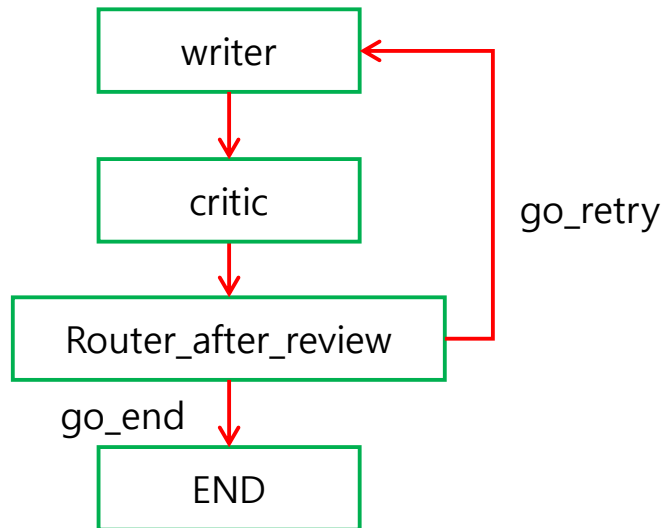


```
workflow.add_edge("billing_agent", "analyzer")  
workflow.add_edge("tech_agent", END)
```

조건부 분기와 루프 구현

루프 구현

- 카피를 작성하도록 하여 원하는 기준에 도달 할때 까지 재작성 하는 로직을 만들어 보자



```
workflow.add_conditional_edges(  
    "critic", # critic 에서 반환한 값으로  
    route_after_review, # 분기를 한다.(go_end|go_retry)  
    {  
        "go_end": END, # 완료 시 종료  
        "go_retry": "writer" # 미흡 시 작가 노드로 되돌아감 (Loop!)  
    }  
)
```

03_loop.py

03.

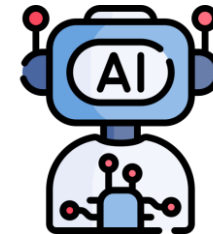
Tool 활용하기

Tool 활용하기

✓ tool

- Tool 은 agent 가 수행해야 하는 일에 사용하는 외부 도구 이다.
- 우리는 langchain 에서 이미 한차례 사용해 보았다.

```
@tool
def multiply(a: int, b: int) -> int:
    """두 수를 곱하는 도구입니다."""
    return a * b
```



이 경우엔 어떤걸
사용 할까?



Tool 활용하기



- LangChain 과 LangGraph 에서 사용하는 tool calling의 차이를 확인해 보자
- 요약하자면 lang chain 의 tool calling 은 로직 수정이 불가능 하거나 제한적이지만 lang graph 는 그렇지 않다.

비교항목	Lang Chain	Lang Graph
제어 흐름	프레임워크 내부 엔진이 흐름을 결정	개발자가 흐름(노드, �지)을 직접 설계
상태 관리	단순히 Messages 만 누적 관리 가능	메시지뿐만 아니라 다양한 커스텀 상태 관리 가능
Loop	내부 엔진에 의한 암묵적 루프만 지원	정교한 순환(Loop) 구조 설계 가능
Human in the loop	불가능하거나 매우 구현하기 까다로움	특정 노드 실행 직전에 일시 중지 및 사람의 피드백 반영이 매우 쉬움
결정론적 가드레일	LLM의 프롬프트 엔지니어링에만 흐름을 의존	하드코딩된 규칙 적용이 쉬움

- * 결정론적: LLM의 기분(확률)에 의존하지 않고 코드로 완벽히 제어함.
- * 가드레일: 시스템이 절대 해서는 안 될 행동이나 정해진 단계의 이탈을 방어함.

Tool 활용하기



- 시작점과 분기 구문을 통해 흐름을 파악해 보자

```
# 시작점 설정
workflow.set_entry_point("agent")

# 에이전트 노드 이후, 라우터를 통해 도구를 실행할지 바로 종료할지 결정
workflow.add_conditional_edges(
    "agent",
    should_continue,
    {
        "call_tool": "action",
        "go_end": END
    }
)

# 도구 실행(action)이 끝나면 다시 에이전트에게 결과를 전달하여 최종 답변을 생성하도록 순환(Loop)을 만듭니다.
workflow.add_edge("action", "agent")
```

04_tools.py

Tool 활용하기



- 간단한 계산기 도구를 활용하는 소스를 만들어 보자
- 먼저 tool 을 등록해 주고 DocString 을 상세하게 적어주자

```
@tool
def multiply(a: int, b: int) -> int:
    """두 정수를 곱하는 계산기 도구입니다.
    곱셈이 필요할 때만 이 도구를 사용하세요.

    Args:
        a: 첫 번째 정수
        b: 두 번째 정수
    """
    print(f"\n[Tool Execution] 'multiply' 도구 실행: {a} * {b}")
    return a * b
```

Tool 활용하기



- 그리고 상태 저장 객체와 툴 등록, LLM 모델 지정도 해 준다.

```
tools = [multiply]
tools_by_name = {tool.name: tool for tool in tools}
```

사용할 툴 등록

```
llm = ChatOllama(model="llama3.1:latest", temperature=0)
llm_with_tools = llm.bind_tools(tools)
```

LLM 모델에 사용 툴 등록

```
class AgentState(TypedDict):
    messages: Annotated[list[BaseMessage], add_messages]
```

BaseMessage 객체가 담기는 리스트
데이터가 들어올때 add_message 라는 함수를 적용한다.

- 이렇게 하는 이유는?

현재 구조는 노드가 값을 반환하면 AgentState 에 들어가게 되는데, Lang Graph 에서는 덮어쓰는것을 원칙으로 한다. 그래서 어노테이션(참고,규칙)을 통해서 overwrite 가 아닌 add message 를 하도록 하는 것이다.

Tool 활용하기

tool

- 사용자가 보낸 메시지를 통해 결과값을 받아온 후 반환 한다.

```
def agent_node(state: AgentState) -> Dict:  
    print("\n[Agent Node] 사용자 메시지 분석 및 다음 행동 결정 중...")  
    response = llm_with_tools.invoke(state["messages"])  
    return {"messages": [response]}
```

- 이후 라우터로 이동하여 다음 경로를 정하게 된다.
- 호출 도구가 있다면 call_tool->action -> tool_node() 로 진행

```
def should_continue(state: AgentState) -> Literal["call_tool", "go_end"]:  
    last_message = state["messages"][-1]  
  
    if hasattr(last_message, "tool_calls") and last_message.tool_calls:  
        print(f"[Router] 도구 호출 감지 -> 'tool_node'로 이동합니다. (호출할 도구: {last_message.tool_calls[0]['name']})")  
        return "call_tool"  
  
    print("[Router] 추가적인 도구 호출 없음 -> 대화를 종료합니다.")  
    return "go_end"
```

Tool 활용하기

```
def tool_node(state: AgentState) -> Dict:  
    print("[Tool Node] 에이전트의 요청에 따라 도구를 실행합니다...")
```

```
    last_message = state["messages"][-1]  
    print(f"last_message : {last_message}")
```

```
    tool_messages = []
```

```
    for tool_call in last_message.tool_calls:  
        tool_name = tool_call["name"]  
        tool_args = tool_call["args"]
```

```
        actual_tool = tools_by_name[tool_name]  
        tool_result = actual_tool.invoke(tool_args)
```

```
        tool_messages.append(  
            ToolMessage(  
                content=str(tool_result),  
                name=tool_name,  
                tool_call_id=tool_call["id"]  
            )  
        )
```

```
    return {"messages": tool_messages}
```

마지막 메시지에는 다음과 같은 속성들을 담고 있다.

HumanMessage: 사용자가 보낸 메시지 (속성: content 등)

ToolMessage: 도구가 반환한 결과 메시지 (속성: content, tool_call_id 등)

AIMessage: LLM(Ollama)이 생성한 응답 메시지 (속성: content, tool_calls 등)

도구 실행 결과를 ToolMessage 형태로 만들어 반환 리스트에 추가

04.

Human in the loop

Human in the loop

정의

- Human in the loop, 인간 개입 가로채기는 진행중인 노드를 정지 시킨 후 계속 진행 여부를 결정하는 기술이다.
- 이때 중지 시 중지 직전 상태를 저장하기 위한 **checkpointer** 옵션이 필요하다.
- 또한 강제 정지시킬 노드를 지정하는 **interrupt_before** 옵션도 필요하다.

```
app = workflow.compile(checkpointer=memory, interrupt_before=["executor"])
```

Human in the loop



로직 진행 지속 여부 선택

- workflow 를 보면 draft 와 executor 두개의 노드가 등록 되어있다.
- draft 를 시작점으로 draft -> executor 로 진행 된다.
- excutor 진행 후에는 종료 된다.
- executor 진행 전에 정지 시켜 사람에게 진행 여부를 확인 받는다.

```
workflow.add_node("drafter", create_draft_node)
workflow.add_node("executor", execute_task_node)
```

```
workflow.set_entry_point("drafter")
workflow.add_edge("drafter", "executor")
workflow.add_edge("executor", END)
```

```
app = workflow.compile(checkpointer=memory, interrupt_before=["executor"])
```

05_human_confirm.py

Human in the loop

☑ 사람 개입으로 Loop 하기

- 이전의 로직은 수락하지 않으면 발송하지 않고 종료 되었다.
- 수락 하지 않으면 재작성을 하는 loop 구조로 변경해 보자

```
class TaskState(TypedDict):  
    task_detail: str  
    title: str # 처음 요청 했던 제목  
    approval: bool
```

수락 여부 상태 값을 추가하자

```
def create_draft_node(state: TaskState) -> Dict:  
    ...(중략)...  
    return {  
        "title": state["title"],  
        "approval": False,  
        "task_detail": content  
    }
```

내용이 계속 변하더라도 제목은 변하면 안되므로 제목 필드를 따로 만들어 준다

06_human_loop.py

Human in the loop

☑ 사람 개입으로 Loop 하기

```
def route_after_draft(state: TaskState)
-> Literal["go_execute", "go_redraft"]:
  if state["approval"] is True:
    return "go_execute"
  else:
    return "go_redraft"
```

```
workflow.add_conditional_edges(
  "drafter",
  route_after_draft,
  {
    "go_execute": "executor",
    "go_redraft": "drafter",
  }
)
```

```
app = workflow.compile(checkpointer=memory, interrupt_after=["drafter"])
```

Drafter 에서 작성된 내용을 본 후 사용자가
선택 하도록 Interrupt_after 사용

조건부 라우터를 추가하여 사용자 선택에 따라 approval 이
변경되고, approval 값에 따라 다른 노드로 움직이도록 설정

Human in the loop

☑ 사람 개입으로 Loop 하기

- 발송이 완료 될 때 까지는 계속해서 순환하며 재 작성된 메일을 확인 받는다.

```
while True:
    user_approval = input("\n작성된 초안을 승인하고 발송하시겠습니까? (Y/N): ")

    if user_approval.strip().upper() == "Y":
        print("\n=== 3단계: 승인 완료로 그래프 재개 ===")
        app.update_state(config,{"approval": True},as_node="drafter")
        app.invoke(None, config)
        break
    else:
        print("\n❌ 발송이 취소되었습니다. 재작성 합니다.")
        app.update_state(config,{"approval": False}, as_node="drafter")
        app.invoke(None, config)
```